

Online supplement:  
A Reproducible Benchmark of Relational  
Database Access-Layer  
Performance in Express.js: A  
Configuration-Specific Comparison  
on PostgreSQL and MySQL

Mateusz Miotk

Faculty of Mathematics, Physics and Informatics, University of Gdańsk  
`mateusz.miotk@ug.edu.pl`

This supplement carries measurement detail referenced from the main text. All tables are generated by the replication package (<https://github.com/mmiothk/express-db-access-performance>; archived at Zenodo as release v1.6.4, <https://doi.org/10.5281/zenodo.21444624>) from the same raw data as the main-text tables.

## 1 Dispersion on MySQL

The main text reports the coefficient-of-variation table for PostgreSQL and the per-engine maxima. Table S1 gives the full per-layer, per-pattern CV on MySQL across the 25 repeated runs (maximum 15.9%, `drizzle`/insert; the insert cells are the noisiest in the study). Figure S1 plots the raw per-replicate insert throughput behind these coefficients: several cells are visibly bimodal or heavy-tailed within a cell, structure a single CV cannot convey, which is why the insert rank correlation and dispersion are reported conservatively.

## 2 Documentation-selected round-trip counts

Table S2 reports, for each layer, the number of SQL statements the documentation-selected deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text’s observation that round-trip count alone does not determine throughput: the two layers issuing four statements (Prisma, Objection) sit at opposite ends of the throughput ladder.

Table S1: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on MySQL. Low values indicate stable measurements.

| Layer     | point_read | range_scan | deep_fetch | aggregation | write |
|-----------|------------|------------|------------|-------------|-------|
| mysql2    | 2.4        | 2.6        | 2.4        | 2.5         | 10.5  |
| knex      | 3.7        | 2.7        | 2.7        | 1.9         | 13.2  |
| drizzle   | 2.9        | 2.2        | 2.0        | 3.5         | 15.9  |
| prisma    | 1.9        | 2.6        | 3.3        | 1.9         | 7.1   |
| sequelize | 2.1        | 1.8        | 2.5        | 1.6         | 12.5  |
| typeorm   | 3.6        | 4.0        | 3.2        | 1.8         | 8.5   |
| objection | 2.5        | 1.9        | 3.8        | 2.9         | 11.7  |
| mikroorm  | 3.0        | 4.0        | 2.2        | 2.1         | 5.6   |

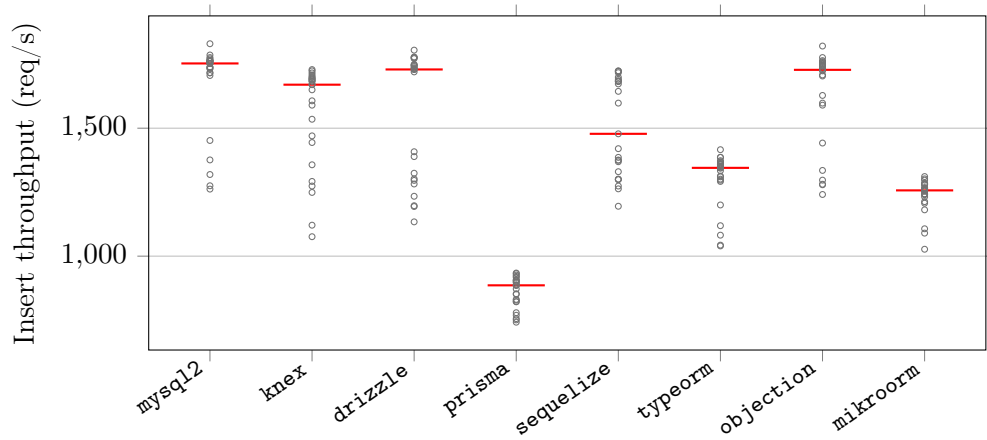


Figure S1: Raw per-replicate insert throughput on MySQL for the eight documentation-selected layers (25 independent runs each, red bar = median). The insert cells are the noisiest in the study; several are visibly bimodal or heavy-tailed within a cell, structure a coefficient of variation cannot convey, which is why the insert rank correlation and dispersion are reported conservatively.

Table S2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs’ eager-loading strategies choose their own counts. Full generated SQL and EXPLAIN plans are in the replication package.

| Layer     | Point read | Range scan | Deep fetch | Aggregation | Insert |
|-----------|------------|------------|------------|-------------|--------|
| pg        | 1          | 1          | 2          | 1           | 1      |
| knex      | 1          | 1          | 2          | 1           | 1      |
| drizzle   | 1          | 1          | 2          | 1           | 1      |
| prisma    | 1          | 1          | 4          | 1           | 1      |
| sequelize | 1          | 1          | 1          | 1           | 1      |
| typeorm   | 1          | 1          | 2          | 1           | 2      |
| objection | 1          | 1          | 4          | 1           | 1      |
| mikroorm  | 1          | 1          | 1          | 1           | 1      |

### 3 Write throughput under default versus relaxed durability

Table S3 gives the per-layer insert throughput under each engine’s default durability configuration (the paper’s primary write regime) and under the symmetric relaxed regime (secondary, 10 replicates), with the relaxed÷default ratio quoted in the main text.

### 4 Equal-CPU-budget sensitivity check

Table S4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores); all nine values are quoted in the main text.

### 5 Application-tier versus end-to-end CPU efficiency

Table S5 reports, per layer on the deep/nested fetch, the median application-process-tree CPU and database-server CPU, the combined core-count, and throughput per application-CPU-second and per combined (application+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database, but on the current versions the application-only and combined figures largely agree, because no layer draws more than about one application core: `mysql2` leads application-tier efficiency by about  $3.8\times$  over Prisma (2,269 vs. 604 req per app-CPU-s) and by  $3.3\times$  end-to-end (1,254 vs. 384 req per combined-CPU-s). There is no CPU-for-throughput trade of the kind an earlier version of this study reported.

Table S3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 10), with the relaxed÷default ratio. At 50 concurrent connections, group commit amortizes the per-commit flushes: relaxing durability buys at most 14% on PostgreSQL and 1.1–2.4× on MySQL, largest for the flush-bound `mysql2` and `Knex` and smallest for the overhead-bound ORMs.

| Layer                     | PostgreSQL |         |       | MySQL   |         |       |
|---------------------------|------------|---------|-------|---------|---------|-------|
|                           | default    | relaxed | ratio | default | relaxed | ratio |
| <code>pg</code>           | 6402       | 7092    | 1.11  | —       | —       | —     |
| <code>mysql2</code>       | —          | —       | —     | 1753    | 4130    | 2.36  |
| <code>pg-tuned</code>     | 6413       | 7340    | 1.14  | —       | —       | —     |
| <code>mysql2-tuned</code> | —          | —       | —     | 1750    | 4039    | 2.31  |
| <code>knex</code>         | 4841       | 5380    | 1.11  | 1670    | 3505    | 2.10  |
| <code>drizzle</code>      | 4085       | 4554    | 1.11  | 1730    | 3280    | 1.90  |
| <code>prisma</code>       | 2774       | 2741    | 0.99  | 886     | 988     | 1.12  |
| <code>sequelize</code>    | 2732       | 2714    | 0.99  | 1478    | 2267    | 1.53  |
| <code>typeorm</code>      | 2648       | 2764    | 1.04  | 1345    | 1647    | 1.22  |
| <code>objection</code>    | 3907       | 4329    | 1.11  | 1728    | 3233    | 1.87  |
| <code>mikroorm</code>     | 1979       | 2040    | 1.03  | 1257    | 1458    | 1.16  |

Table S4: Exploratory equal-CPU sensitivity check (deep-fetch throughput, req/s, PostgreSQL, median of 3 runs, three representative layers) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). `pg` holds full throughput on a single core and gains nothing from more; Prisma and MikroORM stay far below it and do not rise monotonically with extra cores (their per-core values wobble by 10–17% run-to-run at this replicate count), so the deep-fetch ranking is not an artifact of unequal core budgets — a layer’s low per-process throughput is recovered by more processes, not more cores.

| Layer                 | 1 core | 2 cores | 4 cores |
|-----------------------|--------|---------|---------|
| <code>pg</code>       | 3689   | 3688    | 3687    |
| <code>prisma</code>   | 1079   | 1052    | 1219    |
| <code>mikroorm</code> | 446    | 495     | 522     |

Table S5: Application-tier versus end-to-end CPU efficiency on the deep/nested fetch (MySQL): median CPU (% of one logical core) of the Node.js process tree and of the database server, the combined core-count, and throughput per application-CPU-second and per combined (app+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database; the combined figure is the end-to-end view. Load-generator CPU is excluded.

| Layer     | app CPU% | db CPU% | comb. cores | req/app-CPU-s | req/comb-CPU-s |
|-----------|----------|---------|-------------|---------------|----------------|
| mysql2    | 105      | 85      | 1.90        | 2269          | 1254           |
| knex      | 100      | 75      | 1.75        | 2007          | 1147           |
| drizzle   | 105      | 50      | 1.55        | 1255          | 850            |
| prisma    | 105      | 60      | 1.65        | 604           | 384            |
| sequelize | 100      | 30      | 1.30        | 886           | 682            |
| typeorm   | 100      | 70      | 1.70        | 1017          | 598            |
| objection | 100      | 45      | 1.45        | 834           | 575            |
| mikroorm  | 110      | 25      | 1.35        | 436           | 356            |

## 6 Coordinated-omission-corrected open-loop sweep

Table S6 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main text: achieved throughput, corrected latency percentiles measured from each request’s *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

Table S6: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep/nested fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request’s intended start time and timed-out requests included at their clipped value. <sup>†</sup>marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

| Layer     | 250/s | 500/s | 1000/s             | 2000/s             | 4000/s             |
|-----------|-------|-------|--------------------|--------------------|--------------------|
| pg        | 2     | 2     | 2                  | 3                  | 1483 <sup>†</sup>  |
| prisma    | 3     | 3     | 13                 | 10721 <sup>†</sup> | 10947 <sup>†</sup> |
| knex      | 2     | 2     | 2                  | 89                 | 8812 <sup>†</sup>  |
| sequelize | 3     | 3     | 183                | 10745 <sup>†</sup> | 10940 <sup>†</sup> |
| mikroorm  | 4     | 346   | 11283 <sup>†</sup> | 11677 <sup>†</sup> | 11935 <sup>†</sup> |

## 7 Pool-size sensitivity

The primary campaign fixes every layer’s connection pool at 10 to isolate the access layer from pool tuning. Table S7 varies the pool from 1 to 50

for a representative layer of each tier on the deep fetch (PostgreSQL, 50 connections). The deep-fetch ordering (`pg` > `Knex` > `Prisma` > `MikroORM`) is preserved at every pool size, and each layer’s throughput saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking; a very small pool (1–2) is the one regime that compresses the differences, because every layer then queues on connection acquisition.

Table S7: Deep-fetch throughput (req/s, PostgreSQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed-pool choice does not drive the ranking.

| Layer                 | 1   | 2    | 5    | 10   | 20   | 50   |
|-----------------------|-----|------|------|------|------|------|
| <code>pg</code>       | 905 | 1762 | 3224 | 3667 | 3798 | 3768 |
| <code>knex</code>     | 741 | 1413 | 2392 | 2532 | 2683 | 2636 |
| <code>prisma</code>   | 870 | 1072 | 1183 | 1234 | 1207 | 1105 |
| <code>mikroorm</code> | 494 | 487  | 507  | 527  | 527  | 522  |

## 8 Transactional multi-statement write

Beyond the single-row insert of the primary matrix, Table S8 reports a transactional write, `POST /threads`, which inserts a post and five comments in one transaction through each layer’s transaction facility, for a representative layer of each tier on PostgreSQL under default durability. The full layer  $\times$  engine matrix for this pattern is left to future work. Because the transaction commits once, its per-commit `fsync` dominates and the layers cluster more tightly than the widest read spread ( $3.7\times$ , `pg` 2,718 down to `Prisma` 733), though the ordering does not simply track the single-row insert: `Prisma`, mid-pack on the single-row PostgreSQL insert, falls to the bottom of the transactional subset.

## 9 Dispersion on PostgreSQL

Table S9 gives the full per-layer, per-pattern coefficient of variation of throughput on PostgreSQL across the 25 repeated runs (maximum 7.3%, `knex`/insert), the companion to the MySQL table in Section S1.

## 10 Resource footprint

Table S10 gives the per-layer application-process CPU (median % of one logical core) and peak resident memory on the deep/nested fetch (MySQL),

Table S8: Transactional write throughput (req/s) and tail latency (p99, ms) for `POST /threads`, which inserts a post and five comments in a single transaction, on PostgreSQL under default durability (50 connections, median of 5). A representative layer of each tier is shown; the full matrix is left to future work. Between-layer spread is  $3.7\times$  (pg 2718 down to Prisma 733, now the slowest), well below the  $7.15\times$  read spread, and the ordering broadly tracks the single-row insert apart from Prisma (mid-pack single-row, last here), so the write conclusions largely extend to a multi-statement transactional write.

| Layer    | req/s | p99 (ms) |
|----------|-------|----------|
| pg       | 2718  | 22       |
| knex     | 2322  | 27       |
| prisma   | 733   | 80       |
| typeorm  | 1935  | 32       |
| mikroorm | 856   | 68       |

Table S9: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on PostgreSQL. Low values indicate stable measurements.

| Layer     | point_read | range_scan | deep_fetch | aggregation | write |
|-----------|------------|------------|------------|-------------|-------|
| pg        | 4.0        | 2.3        | 3.4        | 2.0         | 5.3   |
| knex      | 4.1        | 3.0        | 2.7        | 3.6         | 7.3   |
| drizzle   | 2.8        | 2.5        | 2.0        | 3.4         | 4.0   |
| prisma    | 3.0        | 2.5        | 3.0        | 2.7         | 3.8   |
| sequelize | 2.6        | 3.0        | 2.2        | 2.2         | 2.6   |
| typeorm   | 2.3        | 2.1        | 2.6        | 3.9         | 3.6   |
| objection | 2.8        | 2.4        | 2.6        | 3.3         | 5.0   |
| mikroorm  | 2.4        | 3.8        | 4.0        | 4.2         | 2.5   |

the detail behind the CPU trade-off discussed in the main text (all layers  $\approx 100\text{--}110\%$  of one core; peak RSS 215–356 MB).

Table S10: Server-process resource footprint on the deep/nested fetch (MySQL): median CPU utilization (% of one core) and peak resident memory (MB) of the Node.js process under load. The load generator and database run in separate processes.

| Layer     | Category        | CPU (%) | RSS (MB) |
|-----------|-----------------|---------|----------|
| mysql2    | native-driver   | 105     | 294      |
| knex      | query-builder   | 100     | 215      |
| drizzle   | orm-lightweight | 105     | 297      |
| prisma    | orm             | 105     | 334      |
| sequelize | orm             | 100     | 262      |
| typeorm   | orm             | 100     | 225      |
| objection | orm             | 100     | 221      |
| mikroorm  | orm             | 110     | 356      |

## 11 Pinned versions

Table S11 lists the pinned versions of every evaluated access layer and tool, recorded from the logfile on the measurement date.

Table S11: Pinned versions of the evaluated access layers and tooling.

| Layer / tool   | Version | Layer / tool    | Version |
|----------------|---------|-----------------|---------|
| pg             | 8.22.0  | sequelize       | 6.37.8  |
| mysql2         | 3.23.0  | typeorm         | 1.1.0   |
| knex           | 3.3.0   | objection       | 3.1.5   |
| drizzle-orm    | 0.45.2  | @mikro-orm/core | 7.1.6   |
| @prisma/client | 7.8.0   | express         | 5.2.1   |
| autocannon     | 8.0.0   | Node.js         | 24.18.0 |
| PostgreSQL     | 18.4    | MySQL           | 9.7.1   |

Pinned to the latest stable release compatible with the harness adapter contract at the freeze date (2026-07-15); Sequelize’s version-7 line is a prerelease, so the 6.x stable line is used. Prisma 7 is Rust-free and connects through an official JS driver adapter (`@prisma/adapters-pg` for PostgreSQL, `@prisma/adapters-mariadb` with the `mariadb` 3.5.3 driver for MySQL, as Prisma ships no `mysql2` adapter).



## 12 Per-pattern detail: the read patterns

The two cheapest read patterns show the smallest between-layer spread and are summarized in the main text; their full throughput and p99 tables are given here. Table S12 reports the point read (primary-key lookup) and Table S13 the keyset range scan, each by access layer and engine. The consequential patterns (deep/nested fetch, aggregation, insert) are tabulated in the main text.

Table S12: Point read: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

| Layer        | Category        | PostgreSQL       |              | MySQL            |              |
|--------------|-----------------|------------------|--------------|------------------|--------------|
|              |                 | req/s [95% CI]   | p99 [95% CI] | req/s [95% CI]   | p99 [95% CI] |
| pg           | native-driver   | 6835 [6597–6892] | 9 [9–10]     | –                | –            |
| mysql2       | native-driver   | –                | –            | 4368 [4314–4420] | 16 [16–16]   |
| pg-tuned     | native-tuned    | 6806 [6622–6964] | 9 [9–9]      | –                | –            |
| mysql2-tuned | native-tuned    | –                | –            | 4268 [4212–4331] | 14 [13–14]   |
| knex         | query-builder   | 4942 [4794–5131] | 13 [12–13]   | 3814 [3771–3870] | 15 [15–16]   |
| drizzle      | orm-lightweight | 4199 [4161–4254] | 14 [14–15]   | 2928 [2866–2977] | 21 [21–22]   |
| prisma       | orm             | 3255 [3222–3310] | 20 [20–21]   | 2074 [2050–2095] | 29 [28–29]   |
| sequelize    | orm             | 3518 [3483–3591] | 17 [16–17]   | 2764 [2726–2798] | 20 [20–21]   |
| typeorm      | orm             | 4185 [4171–4242] | 15 [15–15]   | 3059 [3032–3097] | 19 [19–19]   |
| objection    | orm             | 3977 [3943–4040] | 15 [15–16]   | 3275 [3221–3310] | 17 [17–18]   |
| mikroorm     | orm             | 2455 [2425–2484] | 24 [23–25]   | 2067 [2044–2086] | 27 [26–28]   |

## 13 Same-SQL control: full table

Table S14 gives the full same-SQL (raw-path) control on the deep fetch for both engines: throughput when every layer executes the identical control statement (verified byte-identical by server-side capture). The main text reports the resulting collapse toward parity and reads it as a bound on the residual same-SQL difference, a compound intervention that isolates no single factor.

## 14 Per-pattern detail: aggregation

Table S15 gives the full aggregation-pattern throughput and p99 by access layer and engine. RQ3 finds aggregation the least layer-sensitive pattern once every layer runs the same raw correlated-subquery plan; the main text reports the ranking and the leading figures.

Table S13: Keyset range scan: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

| Layer        | Category        | PostgreSQL       |              | MySQL            |              |
|--------------|-----------------|------------------|--------------|------------------|--------------|
|              |                 | req/s [95% CI]   | p99 [95% CI] | req/s [95% CI]   | p99 [95% CI] |
| pg           | native-driver   | 3811 [3747–3865] | 18 [17–18]   | –                | –            |
| mysql2       | native-driver   | –                | –            | 3295 [3274–3341] | 21 [20–21]   |
| pg-tuned     | native-tuned    | 3830 [3766–3872] | 18 [17–19]   | –                | –            |
| mysql2-tuned | native-tuned    | –                | –            | 3252 [3233–3271] | 19 [17–19]   |
| knex         | query-builder   | 3220 [3198–3240] | 18 [17–19]   | 2995 [2942–3022] | 20 [19–21]   |
| drizzle      | orm-lightweight | 2780 [2749–2822] | 20 [20–21]   | 2165 [2145–2192] | 29 [29–30]   |
| prisma       | orm             | 2443 [2428–2473] | 26 [26–27]   | 1538 [1517–1556] | 39 [38–40]   |
| sequelize    | orm             | 2478 [2424–2498] | 23 [23–24]   | 2105 [2090–2117] | 27 [27–28]   |
| typeorm      | orm             | 2564 [2537–2590] | 24 [24–25]   | 2262 [2150–2298] | 26 [25–27]   |
| objection    | orm             | 2686 [2659–2697] | 22 [21–23]   | 2582 [2551–2597] | 23 [22–23]   |
| mikroorm     | orm             | 905 [891–915]    | 64 [63–68]   | 856 [846–865]    | 66 [66–71]   |

Table S14: Bounding the residual same-SQL difference on the deep-fetch spread: throughput (req/s) of the documentation-selected deep fetch versus the *same-SQL* control (median of 10 runs), in which every layer executes the identical two-statement SQL with identical row mapping through its raw-SQL facility. The ratio (default ÷ same-SQL) measures the gap between each layer’s documentation-selected relational-fetch path and the common raw-SQL path, a composite of query strategy, query count, protocol, the raw versus ORM API, and result marshalling and hydration changed together; the residual native-relative same-SQL spread ( $1.68\times$  on PostgreSQL,  $2.01\times$  on MySQL) bounds that compound difference without isolating any single factor.

| Layer     | PostgreSQL |          |       | MySQL   |          |       |
|-----------|------------|----------|-------|---------|----------|-------|
|           | default    | same-SQL | ratio | default | same-SQL | ratio |
| pg        | 3687       | 3625     | 1.02  | –       | –        | –     |
| mysql2    | –          | –        | –     | 2382    | 2427     | 0.98  |
| knex      | 2487       | 2926     | 0.85  | 2007    | 2245     | 0.89  |
| drizzle   | 1865       | 3237     | 0.58  | 1318    | 2008     | 0.66  |
| prisma    | 1186       | 2155     | 0.55  | 634     | 1206     | 0.53  |
| sequelize | 991        | 2436     | 0.41  | 886     | 1903     | 0.47  |
| typeorm   | 1204       | 3170     | 0.38  | 1017    | 2267     | 0.45  |
| objection | 1028       | 2831     | 0.36  | 834     | 2310     | 0.36  |
| mikroorm  | 516        | 3239     | 0.16  | 480     | 2453     | 0.20  |

Table S15: Aggregation: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

| Layer        | Category        | PostgreSQL       |              | MySQL            |              |
|--------------|-----------------|------------------|--------------|------------------|--------------|
|              |                 | req/s [95% CI]   | p99 [95% CI] | req/s [95% CI]   | p99 [95% CI] |
| pg           | native-driver   | 6978 [6865–7038] | 11 [10–11]   | –                | –            |
| mysql2       | native-driver   | –                | –            | 3994 [3941–4018] | 18 [17–19]   |
| pg-tuned     | native-tuned    | 7538 [7493–7705] | 10 [9–10]    | –                | –            |
| mysql2-tuned | native-tuned    | –                | –            | 3958 [3933–3985] | 15 [15–17]   |
| knex         | query-builder   | 5666 [5608–5710] | 13 [12–14]   | 3842 [3815–3883] | 19 [18–20]   |
| drizzle      | orm-lightweight | 6280 [6156–6427] | 12 [12–12]   | 3520 [3447–3541] | 21 [19–21]   |
| prisma       | orm             | 4408 [4368–4476] | 16 [15–16]   | 2328 [2301–2349] | 30 [28–31]   |
| sequelize    | orm             | 4813 [4758–4870] | 14 [14–15]   | 3216 [3203–3235] | 20 [18–22]   |
| typeorm      | orm             | 6098 [5985–6179] | 13 [12–13]   | 3767 [3731–3812] | 18 [15–20]   |
| objection    | orm             | 3812 [3724–3851] | 17 [17–18]   | 2763 [2743–2785] | 25 [23–26]   |
| mikroorm     | orm             | 5745 [5637–5805] | 13 [12–14]   | 3741 [3711–3804] | 19 [18–21]   |

## 15 Documentation-selected deep-fetch choices per access layer

Table S16 documents, for each access layer, the exact eager-loading API used on the deep/nested fetch, the number of SQL round trips it issues, the documented alternative loading strategy we did *not* use, and the query-logging state. Every layer uses its documented default path; query logging is disabled and verified on all eleven adapters, and no lifecycle hooks or validation run on the read path. The six data-mapper and ORM layers split both ways on the join-versus-select-in default: Sequelize, TypeORM, and MikroORM default to a single join, while Prisma and Objection default to separate batched queries. The loading-strategy sensitivity check reported in the main text (Table S18) switches the join-default layers to select-in where that alternative is a byte-identical drop-in. Full per-adapter detail is in the replication package’s `METHODOLOGY.md`.

## 16 Open-loop tail latency on MySQL

Table S17 is the MySQL companion to the PostgreSQL open-loop sweep (Supplement Table S6): coordinated-omission-corrected p99 on the deep/nested fetch at a constant offered rate. The sub-saturation tail is flat and small on both engines, and each layer collapses once the offered rate crosses its capacity, so the high-load-tail ordering holds on MySQL as on PostgreSQL.

Table S16: Documentation-selected deep-fetch implementation per access layer: the eager-loading API actually used, the SQL round trips it issues (measured from server-side statement logging for the portable layers; by construction for the native variants), the documented alternative loading strategy we did *not* use, and the query-logging state. Query logging is disabled everywhere; no lifecycle hooks or validation run on the read path (details in the replication package’s `METHODOLOGY.md`).

| Layer        | Deep-fetch API                  | Round-trips | Alternative not used            | Logging |
|--------------|---------------------------------|-------------|---------------------------------|---------|
| pg           | hand-written JOIN $\times 2$    | 2           | n/a (hand-written SQL)          | off     |
| mysql2       | hand-written JOIN $\times 2$    | 2           | n/a (hand-written SQL)          | off     |
| pg-tuned     | named-prepared JOIN $\times 2$  | 2           | n/a (hand-written SQL)          | off     |
| mysql2-tuned | execute() JOIN $\times 2$       | 2           | n/a (hand-written SQL)          | off     |
| knex         | builder .join() $\times 2$      | 2           | n/a (hand-written SQL)          | off     |
| drizzle      | builder .innerJoin() $\times 2$ | 2           | relational query API (db.query) | off     |
| prisma       | include:                        | 4           | relationLoadStrategy: 'join'    | off     |
| sequelize    | include: (separate:false)       | 1           | separate:true (select-in)       | off     |
| typeorm      | relations:                      | 2           | relationLoadStrategy: 'query'   | off     |
| objection    | .withGraphFetched()             | 4           | .withGraphJoined()              | off     |
| mikroorm     | populate:                       | 1           | strategy: 'select-in'           | off     |

Table S17: Open-loop tail latency on **MySQL**, corrected for coordinated omission: p99 (ms) on the deep/nested fetch under a constant offered request rate, every latency measured from the intended start time and timed-out requests clipped into the distribution. <sup>†</sup>marks cells that did not sustain the offered rate. As on PostgreSQL (Supplement Table S6), the sub-saturation tail is flat and small and each layer collapses once the offered rate crosses its capacity, so the high-load-tail ordering is engine-robust.

| Layer     | 250/s | 500/s            | 1000/s             | 2000/s             | 4000/s             |
|-----------|-------|------------------|--------------------|--------------------|--------------------|
| mysql2    | 2     | 2                | 3                  | 15                 | 10856 <sup>†</sup> |
| prisma    | 5     | 7                | 9518 <sup>†</sup>  | 11723 <sup>†</sup> | 11900 <sup>†</sup> |
| knex      | 2     | 2                | 2                  | 243                | 10904 <sup>†</sup> |
| sequelize | 3     | 3                | 1628 <sup>†</sup>  | 10797 <sup>†</sup> | 10962 <sup>†</sup> |
| mikroorm  | 4     | 953 <sup>†</sup> | 11314 <sup>†</sup> | 11788 <sup>†</sup> | 11912 <sup>†</sup> |

## 17 Alternative eager-loading sensitivity

Table S18 reports the loading-strategy sensitivity check: for the data-mapper ORMs whose alternative strategy is a byte-identical drop-in, documentation-selected deep-fetch throughput versus the alternative, on both engines. Both endpoints were verified to return byte-identical responses before timing.

Table S18: Alternative eager-loading sensitivity on the deep/nested fetch: documentation-selected throughput (req/s) versus the alternative loading strategy, both engines, for the layers whose alternative is a byte-identical drop-in. Sequelize and MikroORM switch from their documentation-selected single join to select-in; in both cases the documentation-selected default is the *faster* strategy, so their deep-fetch deficit is not an artifact of a poor default. TypeORM is omitted, its alternative strategy not a byte-identical drop-in in the pinned versions (its query strategy errors on the ordered nested `findOne`, and Objection’s `withGraphJoined` changes row handling), which is itself a portability caveat.

| Layer                  | Switch         | PostgreSQL |      | MySQL   |      |
|------------------------|----------------|------------|------|---------|------|
|                        |                | default    | alt. | default | alt. |
| <code>sequelize</code> | join→select-in | 915        | 690  | 830     | 602  |
| <code>mikroorm</code>  | join→select-in | 447        | 357  | 420     | 324  |
| <code>objection</code> | select-in→join | —          | —    | 820     | 1301 |

## 18 MySQL insert commit-flush mechanism

Table S19 gives the direct `performance_schema` evidence behind the MySQL insert bottleneck: sampling the redo-log and binary-log flush wait instruments around a fixed insert workload at default durability, the flush is a shared engine floor—similar per-insert across the native driver, query builder, and ORM despite their very different read performance—and relaxing durability removes it.

## 19 Post-restart environmental-robustness check

Table S20 reports the environmental-robustness check: as the campaign’s final stage, after a full restart of both database engines (cold buffer pools, fresh connections), a representative deep-fetch subset was re-measured against the primary median. The layer ranking reproduces on both engines (`pg` > Prisma  $\gg$  MikroORM on PostgreSQL; `mysql2`  $\gg$  Prisma > MikroORM on MySQL); absolute throughput is up to 16% lower, a cold-buffer and run-to-run drift the paper’s relative analysis is designed to absorb. Because PostgreSQL and MySQL

Table S19: MySQL insert commit-flush mechanism (`performance_schema` wait instruments, default durability `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`): per-insert wall time, the redo-log+binlog flush wait per insert, and the flush share of wall time, over 4000 inserts per layer. The flush wait is a shared engine floor – similar across layers despite their very different read performance – so it caps insert throughput regardless of the access layer. Relaxing durability (`mysql2`, `trx_commit=0`, `sync_binlog=0`) removes the flush and lifts throughput to 3238 req/s, confirming the floor.

| Layer (MySQL)                        | req/s | per-insert (ms) | flush/insert (ms) | flush share |
|--------------------------------------|-------|-----------------|-------------------|-------------|
| <code>mysql2</code>                  | 1576  | 0.634           | 1.001             | 158%        |
| <code>knex</code>                    | 1722  | 0.581           | 1.009             | 174%        |
| <code>mikroorm</code>                | 2584  | 0.387           | 0                 | 0%          |
| <i>Relaxed durability (control):</i> |       |                 |                   |             |
| <code>mysql2*</code>                 | 3238  | 0.309           | 0.01              | 3%          |

cells are not measured simultaneously, this drift is non-uniform across engines (0% on `mysql2`, 16% on `pg`, 16% on MikroORM/PostgreSQL), so the cross-engine interaction ratios (Table S28) could in principle carry some residual drift; the engine effects there are far larger than the 16% drift, so the engine-dependence conclusion holds.

Table S20: Environmental-robustness check (review 6.7): as the campaign’s final stage, after a full restart of both database engines (cold buffer pools, fresh connections), deep-fetch throughput of a representative subset was re-measured against the primary median. The layer ranking reproduces on both engines; absolute throughput drifts with host conditions (larger on PostgreSQL), a cold-buffer and run-to-run variation the paper’s *relative* analysis is designed to absorb. Single host, so this checks restart robustness, not cross-machine generalization.

| Engine     | Layer                 | primary req/s | post-restart req/s | ratio |
|------------|-----------------------|---------------|--------------------|-------|
| PostgreSQL | <code>pg</code>       | 3687          | 3100.5             | 0.84  |
| PostgreSQL | <code>prisma</code>   | 1186          | 1153               | 0.97  |
| PostgreSQL | <code>mikroorm</code> | 516           | 434.5              | 0.84  |
| MySQL      | <code>mysql2</code>   | 2382          | 2375.5             | 1.00  |
| MySQL      | <code>prisma</code>   | 634           | 618.5              | 0.98  |
| MySQL      | <code>mikroorm</code> | 480           | 437.5              | 0.91  |

## 20 Utilization-controlled tail latency

Tables S21 and S22 give the coordinated-omission-corrected p99 on the deep fetch when each layer is offered 50/70/85/95% of *its own* saturating throughput (its primary closed-loop deep-fetch rate), replicated five times, on both engines. At matched utilization the tails are small and similar across layers—for example on PostgreSQL every layer holds p99 near 2–4 ms at 50%—so the wide high-load p99 ladder in the main text is a capacity-and-queueing effect, not a difference in tail latency at matched utilization; the tail rises only as a layer nears its own capacity. This is the capacity-normalized latency companion to the high-load primary p99. The 50/70% cells, which carry that comparison, are stable across runs; the 85/95% cells are increasingly noisy (p99 varies several-fold between runs as a layer approaches saturation), so those columns are read as trend, not precise values.

Table S21: Utilization-controlled open-loop tail on the deep fetch (PostgreSQL): coordinated-omission-corrected p99 (ms, median of five runs) when each layer is offered a fixed fraction of *its own* saturating throughput. At matched utilization the tails are small and similar across layers—the large high-load p99 gap is a capacity/queueing effect, not intrinsic tail latency; the tail rises only as each layer approaches its own capacity.

| Layer     | 50% | 70% | 85%  | 95%   |
|-----------|-----|-----|------|-------|
| pg        | 3.9 | 3.1 | 4    | 5.9   |
| knex      | 2.4 | 3.2 | 3.5  | 54    |
| drizzle   | 3.8 | 8.8 | 30.9 | 7.4   |
| prisma    | 4.2 | 7.8 | 18.8 | 77.5  |
| sequelize | 2.5 | 4.3 | 7.7  | 11.8  |
| typeorm   | 3.1 | 3.7 | 7.9  | 37.1  |
| objection | 3.1 | 3.1 | 6.2  | 284.6 |
| mikroorm  | 3.9 | 4.9 | 5.5  | 7.5   |

## 21 Longer-window tail sensitivity

The primary p99 is measured over a 12 s window, which yields only  $\approx 60$  requests above the p99 threshold in the slowest deep-fetch cells. To check that this per-run tail count is sufficient, Table S23 re-measures the deep fetch over a 60 s window (about five times the requests,  $\approx 300$  tail observations per run) at the same 50-connection operating point, ten independent runs, on both engines. The p99 ranking is preserved exactly (Spearman  $\rho = 1.00$  per engine), each cell’s p99 barely moves from the 12 s value, the p97.5 ordering matches the p99 ordering ( $\rho = 1.00$ ), and the run-to-run p99 coefficient

Table S22: Utilization-controlled open-loop tail on the deep fetch (MySQL): coordinated-omission-corrected p99 (ms, median of five runs) when each layer is offered a fixed fraction of *its own* saturating throughput. At matched utilization the tails are small and similar across layers—the large high-load p99 gap is a capacity/queueing effect, not intrinsic tail latency; the tail rises only as each layer approaches its own capacity.

| Layer     | 50% | 70% | 85%  | 95%  |
|-----------|-----|-----|------|------|
| mysql2    | 4.2 | 7.7 | 20.9 | 39.7 |
| knex      | 1.9 | 2.7 | 13.7 | 4.1  |
| drizzle   | 3.7 | 6.1 | 14.9 | 17.6 |
| prisma    | 5.2 | 5.6 | 11.9 | 51.4 |
| sequelize | 2.5 | 3.2 | 6.8  | 21   |
| typeorm   | 2.8 | 3.6 | 3    | 36.5 |
| objection | 2.9 | 3.2 | 3.7  | 32.9 |
| mikroorm  | 4   | 4.2 | 8.7  | 31.2 |

of variation stays small (maximum 6.0% on PostgreSQL, 3.8% on MySQL). The shorter window is therefore adequate for the tail *ranking*, and the tail conclusions do not rest on the exact per-run tail count.

## 22 Multi-worker scaling

Table S24 scales each of a representative subset to 1, 2, and 4 Express workers (a shared-port `node` cluster). Because each process uses about one core and gains nothing from more, throughput scales roughly linearly with workers until the host saturates: on PostgreSQL the native driver leads at one worker (pg 3,299 to Prisma’s 1,117) and saturates near two workers as it becomes database-bound (4,055, then 3,961 at four), while Prisma, low per process, rises 1,117 to 2,275 to 4,449 and reaches native-like aggregate throughput at four workers. A layer’s low per-process throughput is therefore recoverable by horizontal scaling, at a proportional cost in cores—the direct test of the single-host resource-competition concern.

## 23 Pool-size frontier on MySQL

Table S25 is the MySQL companion to the PostgreSQL pool sweep (Supplement Table S7): deep-fetch throughput as the connection pool varies from 1 to 50, per layer. Each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed pool of ten does not drive the ranking on either engine.



Table S23: Longer-window tail sensitivity on the deep/nested fetch. Each cell’s primary p99 is measured over a 12 s window ( $\approx 60$  tail observations per run in the slowest cells); the re-measurement uses a 60 s window ( $\approx 300$  tail observations) at the same 50-connection operating point, 10 independent runs. The p99 ranking is preserved exactly (Spearman  $\rho = 1.00$  on both engines), the per-cell p99 barely moves, the p97.5 ordering matches the p99 ordering ( $\rho = 1.00$ ), and the run-to-run p99 CV stays small (max 6.0% PostgreSQL, 3.8% MySQL), so the 12 s window is adequate for the tail ranking despite the modest per-run tail count.

| Layer             | p99 12 s<br>(ms) | p99 60 s<br>(ms) | p97.5 60 s<br>(ms) | p99 CV<br>(%) | rps 60 s |
|-------------------|------------------|------------------|--------------------|---------------|----------|
| <i>PostgreSQL</i> |                  |                  |                    |               |          |
| pg-tuned          | 18               | 18               | 17                 | 2.3           | 3785.5   |
| pg                | 20               | 20.5             | 18.5               | 6.0           | 3687.5   |
| knex              | 27               | 28               | 26                 | 2.9           | 2460     |
| drizzle           | 35               | 35               | 33                 | 2.3           | 1855.5   |
| typeorm           | 50               | 51               | 49                 | 3.7           | 1222     |
| prisma            | 51               | 52.5             | 50                 | 3.1           | 1176.5   |
| objection         | 57               | 59               | 56.5               | 2.9           | 1026     |
| sequelize         | 60               | 60               | 57                 | 1.5           | 994      |
| mikroorm          | 116              | 112.5            | 109                | 2.8           | 519.5    |
| <i>MySQL</i>      |                  |                  |                    |               |          |
| mysql2-tuned      | 25               | 25.5             | 24                 | 3.3           | 2409.5   |
| mysql2            | 28               | 28               | 25.5               | 2.6           | 2443.5   |
| knex              | 30               | 29.5             | 28                 | 3.3           | 2048     |
| drizzle           | 47               | 48               | 45                 | 3.2           | 1297     |
| typeorm           | 57               | 58.5             | 56                 | 1.8           | 1015.5   |
| sequelize         | 67               | 66               | 63                 | 2.2           | 889      |
| objection         | 69               | 70               | 67.5               | 2.1           | 845.5    |
| prisma            | 93               | 95               | 92                 | 3.2           | 630      |
| mikroorm          | 120              | 123.5            | 119                | 3.8           | 478.5    |

Table S24: Multi-worker deep-fetch throughput (req/s, median of three runs) as each layer is scaled to 1, 2, and 4 Express workers (node cluster, shared port). Because each process uses about one core, throughput scales roughly linearly with workers until the host saturates: Prisma, low at one worker (PostgreSQL 1,117), rises to 2,275 at two and 4,449 at four workers, recovering native-like aggregate throughput at four times the processes. A layer’s low per-process throughput is therefore recoverable by horizontal scaling, at a proportional cost in cores.

| Layer             | 1 worker | 2 workers | 4 workers |
|-------------------|----------|-----------|-----------|
| <i>PostgreSQL</i> |          |           |           |
| pg                | 3299     | 4055      | 3961      |
| prisma            | 1117     | 2275      | 4449      |
| mikroorm          | 435      | 898       | 1773      |
| <i>MySQL</i>      |          |           |           |
| mysql2            | 2327     | 4698      | 8883      |
| prisma            | 591      | 1167      | 2427      |
| mikroorm          | 418      | 851       | 1624      |

Table S25: Deep-fetch throughput (req/s, MySQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed-pool choice does not drive the ranking.

| Layer    | 1    | 2    | 5    | 10   | 20   | 50   |
|----------|------|------|------|------|------|------|
| mysql2   | 1545 | 2156 | 2352 | 2446 | 2506 | 2494 |
| knex     | 1255 | 1926 | 1984 | 2081 | 2131 | 2120 |
| prisma   | 464  | 572  | 631  | 639  | 626  | 648  |
| mikroorm | 416  | 469  | 486  | 418  | 492  | 429  |

## 24 Mixed read/write workload

Table S26 interleaves the deep-fetch read with single-row inserts at 90:10 and 70:30 read:write, both engines, with a physical write reset per run. The layer ordering matches the read ordering and is insensitive to the mix, because the deep-fetch read dominates the cost; exploratory.

Table S26: Mixed read/write workload: throughput (req/s) and p99 (ms) under an autocannon request mix interleaving the deep-fetch read with single-row inserts, at 90:10 and 70:30 read:write, both engines, median of five runs with a physical write reset per run. The layer ordering matches the read ordering and is insensitive to the mix (the deep-fetch read dominates); exploratory.

| Engine | Layer    | read:write | req/s | p99 |
|--------|----------|------------|-------|-----|
| PG     | pg       | 90:10      | 4236  | 19  |
| PG     | pg       | 70:30      | 4347  | 18  |
| PG     | knex     | 90:10      | 3024  | 26  |
| PG     | knex     | 70:30      | 2993  | 26  |
| PG     | prisma   | 90:10      | 1504  | 55  |
| PG     | prisma   | 70:30      | 1409  | 58  |
| PG     | mikroorm | 90:10      | 676   | 103 |
| PG     | mikroorm | 70:30      | 705   | 99  |
| MySQL  | mysql2   | 90:10      | 2283  | 44  |
| MySQL  | mysql2   | 70:30      | 2505  | 47  |
| MySQL  | knex     | 90:10      | 2275  | 37  |
| MySQL  | knex     | 70:30      | 2232  | 40  |
| MySQL  | prisma   | 90:10      | 718   | 114 |
| MySQL  | prisma   | 70:30      | 729   | 103 |
| MySQL  | mikroorm | 90:10      | 626   | 109 |
| MySQL  | mikroorm | 70:30      | 639   | 105 |

## 25 Concurrency sweep

Figure S2 sweeps the deep/nested fetch from 1 to 256 client connections for a representative layer of each tier, both engines. The 50-connection operating point used throughout sits above the knee for every layer on this pattern; the ordering is load-robust above about 32 connections. This is the evidence behind the choice of operating point.

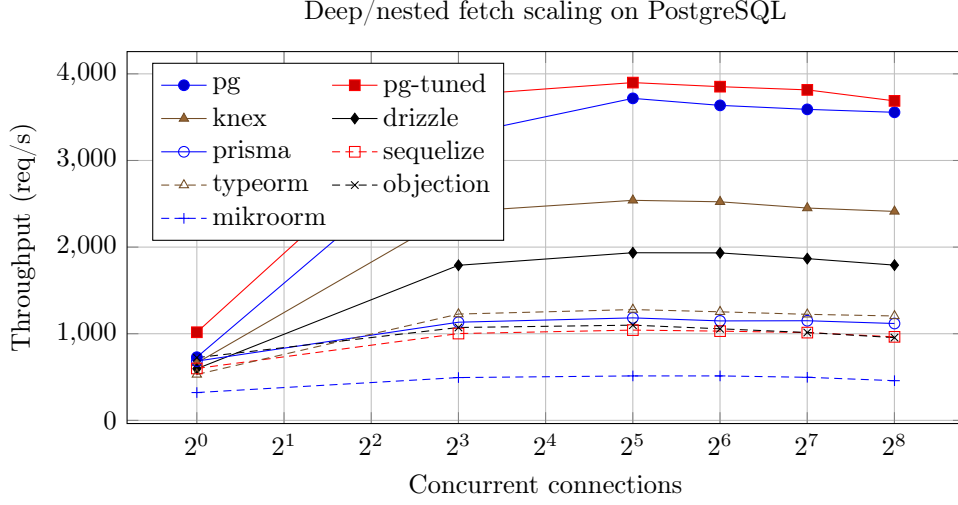


Figure S2: Throughput of each access layer on the deep/nested fetch as concurrency increases (postgres). The native driver leads at every concurrency level, with the query builder and lightweight ORM Drizzle next and the data-mapper ORMs (Prisma among them) below; the ordering is stable above about 32 connections and compresses only at very low concurrency.

## 26 Per-layer cross-engine ranks

Table S27 gives each portable layer’s throughput rank on PostgreSQL versus MySQL for the deep fetch and the insert, backing the rank correlations in the main text: the deep-fetch orderings largely agree ( $\rho = 0.86$ ) while the insert orderings scramble more ( $\rho = 0.68$ ), most visibly Prisma (rank 4 on PostgreSQL, 7 on MySQL).

Because MikroORM is the largest single outlier in several spreads, we recompute the cross-engine agreement without it: over the remaining six portable layers the read ordering stays similar (deep-fetch  $\rho = 0.77$ ,  $\tau = 0.60$ ; range  $\rho = 0.83$ ,  $\tau = 0.73$ ) and the deep-fetch native-relative spread stays substantial at  $3.72\times$  (PostgreSQL) and  $3.76\times$  (MySQL), so the RQ2 and RQ3 findings do not depend on that one layer.

## 27 Layer-by-engine interaction magnitude

The blocked permutation test reports *that* the layer ordering interacts with the engine; Table S28 reports *how large* the interaction is, as the PostgreSQL÷MySQL throughput ratio per layer and pattern with a 95% paired-bootstrap interval. Every layer is faster on PostgreSQL, so the interaction is in the spread of the advantage: the reads stay within  $\approx 1.0$ – $1.9$  while the insert scatters from 1.6 to 3.2, the quantitative counterpart of the low insert rank correlation.

Table S27: Per-layer throughput rank (1 = fastest) on PostgreSQL versus MySQL for the seven portable layers, backing the rank correlations. On the deep/nested fetch the two engines’ orderings largely agree (Spearman  $\rho = 0.86$ , Kendall  $\tau = 0.71$ ); on the insert they scramble more ( $\rho = 0.68$ ,  $\tau = 0.52$ ), most visibly Prisma, which is mid-pack on PostgreSQL (rank 4) but last on MySQL (rank 7). This is why aggregation and writes are reported as transferring only moderately across engines. Excluding MikroORM (the largest single outlier) leaves the read ordering similar (deep-fetch  $\rho = 0.77$ ,  $\tau = 0.60$ ) and the deep-fetch native-relative spread at  $3.72\times$  (PostgreSQL) and  $3.76\times$  (MySQL), so no single layer drives these findings.

| Layer            | Deep fetch |       | Insert |       |
|------------------|------------|-------|--------|-------|
|                  | PG         | MySQL | PG     | MySQL |
| <b>knex</b>      | 1          | 1     | 1      | 3     |
| <b>drizzle</b>   | 2          | 2     | 2      | 1     |
| <b>prisma</b>    | 4          | 6     | 4      | 7     |
| <b>sequelize</b> | 6          | 4     | 5      | 4     |
| <b>typeorm</b>   | 3          | 3     | 6      | 5     |
| <b>objection</b> | 5          | 5     | 3      | 2     |
| <b>mikroorm</b>  | 7          | 7     | 7      | 6     |

Table S28: Layer  $\times$  engine interaction magnitude: the PostgreSQL  $\div$  MySQL throughput ratio (geometric mean of the paired per-replicate ratios, with a 95% paired-bootstrap interval) for each portable layer and pattern. A ratio  $> 1$  means the layer runs faster on PostgreSQL,  $< 1$  faster on MySQL. Every layer is faster on PostgreSQL here, so the interaction is in the *spread* of the advantage, not its sign: point read and range scan hold a tight band ( $\approx 1.0$ – $1.6$ ), the deep fetch and aggregation reach  $\approx 1.9$ , and the insert scatters widest ( $1.6$ – $3.2$ ), reordering the layers across engines — the interaction the blocked permutation test detects and the rank correlations of Table S27 summarize.

| Layer            | Point read       | Range scan       | Deep fetch       | Aggregation      | Insert           |
|------------------|------------------|------------------|------------------|------------------|------------------|
| <b>knex</b>      | 1.31 [1.28,1.34] | 1.07 [1.06,1.09] | 1.23 [1.21,1.25] | 1.46 [1.44,1.48] | 3.05 [2.89,3.23] |
| <b>drizzle</b>   | 1.44 [1.42,1.46] | 1.28 [1.27,1.30] | 1.41 [1.40,1.42] | 1.80 [1.77,1.83] | 2.62 [2.46,2.79] |
| <b>prisma</b>    | 1.57 [1.55,1.59] | 1.59 [1.57,1.62] | 1.89 [1.85,1.92] | 1.90 [1.87,1.92] | 3.23 [3.14,3.32] |
| <b>sequelize</b> | 1.28 [1.26,1.29] | 1.17 [1.15,1.19] | 1.12 [1.10,1.13] | 1.49 [1.48,1.51] | 1.82 [1.73,1.91] |
| <b>typeorm</b>   | 1.38 [1.36,1.41] | 1.15 [1.12,1.17] | 1.19 [1.17,1.21] | 1.60 [1.57,1.63] | 2.05 [1.97,2.13] |
| <b>objection</b> | 1.22 [1.20,1.24] | 1.04 [1.03,1.05] | 1.22 [1.21,1.24] | 1.37 [1.35,1.40] | 2.38 [2.28,2.50] |
| <b>mikroorm</b>  | 1.19 [1.17,1.21] | 1.06 [1.04,1.07] | 1.06 [1.04,1.08] | 1.51 [1.48,1.54] | 1.60 [1.56,1.64] |

## 28 A reporting checklist for access-layer benchmarks

Distilled from the design decisions and the pitfalls this study confronted, the following items are the ones an access-layer benchmark should report so that its claims can be scoped and reproduced. It is a reporting aid, not a standard.

1. **Correctness before timing.** Assert byte-identical responses across layers (final serialized bodies), or state what differs; several defects in this study were caught only this way.
2. **Experimental unit.** Treat the freshly booted run, not the individual request, as the unit; requests within a run are not independent.
3. **Same-host repetition is not environmental replication.** Report how many runs, on how many hosts, over what period; do not call same-host repeats “independent.”
4. **Documentation-selected treatment.** State the exact API, its documentation page and version, the declined alternative, and whether logging, hooks, validation, and identity maps are on.
5. **Latency construct.** Distinguish high-load closed-loop response time (which tracks throughput and queueing) from utilization-controlled or open-loop latency; report the operating point.
6. **Same-SQL / shared-plan control.** Report it as a compound intervention (it changes query formulation, protocol, preparation, round-trips, and mapping together), not a mechanism decomposition.
7. **Resource accounting.** Report application-tier and database CPU separately; throughput per CPU-second, not throughput alone.
8. **Write-state control.** Reset the mutable state between runs and state the durability regime (default versus relaxed).
9. **Provenance.** Pin engine images by digest and dependencies by lock-file; archive raw per-cell records, seeds, and a table-to-record manifest.
10. **Scope.** State the host topology, workload, versions, and dates, and confine claims to them.

## 29 Study factors

Table [S29](#) records which factors the benchmark holds constant, which it varies, and which are uncontrolled on the single virtualized host; the uncontrolled factors are why results are reported for this configuration rather than as a general library ranking.

Table S29: Factors in the benchmark: what is held constant, what varies (the treatments), and what is uncontrolled. The same-SQL control additionally holds the generated SQL and row mapping constant; because it changes query formulation, API, protocol, statement preparation, round-trip structure, and mapping *together*, it bounds the residual difference but does not isolate any single factor. The uncontrolled factors are inherent to a single virtualized host and are the reason results are reported for this configuration, not as general library performance.

| Status              | Factor                            | Detail                                                     |
|---------------------|-----------------------------------|------------------------------------------------------------|
| Held constant       | schema, indexes, seed data        | identical DDL + deterministic seed, both engines           |
|                     | per-replicate request stream      | paired PRNG stream, seeded on (endpoint, replicate)        |
|                     | connection pool                   | fixed at ten for every layer                               |
|                     | response bytes                    | byte-identical JSON across layers (verified)               |
|                     | host, OS, engine versions         | one host; pinned July-2026 versions (Supplement Table S11) |
|                     | warm-up, run duration, cell order | 15s warm-up, 12s runs, randomized order                    |
| Varies (treatments) | access layer                      | 9 documentation-selected + 2 tuned implementations         |
|                     | database engine                   | PostgreSQL, MySQL                                          |
|                     | access pattern                    | point read, range scan, deep fetch, aggregation, insert    |
| Uncontrolled        | hypervisor neighbours, CPU steal  | single virtualized host                                    |
|                     | thermal / frequency scaling       | not pinned; bounded by the drift and post-restart checks   |
|                     | database cache / kernel history   | warm by design; randomized order + fresh boot mitigate     |

## 30 Paired significance of the p99 ladder

Table S30 is the p99 counterpart of the main-text throughput significance table (Table 6): the same paired permutation, Wilcoxon, geometric-mean-ratio, paired-bootstrap-CI, and dominance analysis applied to each adjacent layer’s per-replicate deep-fetch p99 on both engines, delivering the per-cell p99 inference the outcomes table (Table 3) lists as a primary analysis. Every adjacent pair separates on both engines except two near-ties at ratio 1.03 — the TypeORM–Prisma step on PostgreSQL (permutation  $p = 0.05$ ) and the Sequelize–Objection step on MySQL ( $p = 0.08$ ); the p99 ordering otherwise mirrors the throughput ranking. Because the high-load tail tracks capacity, these are resolved capacity-and-queueing differences, not intrinsic per-request latency.

## 31 Benchmarking-pitfalls checklist

The four confounds summarized in the main text’s Discussion, in full. Each is a generic failure mode of the genre, easy to introduce by accident and hard to detect: the correctness ones without an automated cross-check, the performance ones without profiling every cell.

- **Aggregation fan-out.** A naive join between `posts` and `comments` evaluated before the `SUM` multiplies each post’s view count by its number of matching comment rows, inflating the aggregate — a correctness

Table S30: Paired comparison of adjacently ranked access layers on the deep/nested fetch by response-time **p99** — the p99 counterpart to the throughput significance table (Table 6 in the main text), over the 25 repeated runs on each engine. Layers are ranked by median p99 (lower is better) and each adjacent pair is compared on its per-replicate p99 ratios: median p99 (ms) with a within-campaign bootstrap 95% CI, the geometric-mean paired ratio B/A (how many times the slower-tail layer’s p99 exceeds the faster’s) with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which B’s p99 exceeds A’s), and the two-sided paired sign-flip permutation  $p$  (20000 permutations;  $5.0 \times 10^{-5}$  is the resolution floor  $1/(B+1)$ , and the Wilcoxon signed-rank test agrees, replication package). This delivers the per-cell p99 inference the outcomes table (Table 3 in the main text) lists as a primary analysis; because the high-load tail tracks capacity, these are resolved capacity-and-queueing differences, not intrinsic per-request latency.

| Comparison (A < B, p99) | med. A p99 [95% CI] | med. B p99 [95% CI] | ratio B/A [95% CI] | dom. | perm. $p$            |
|-------------------------|---------------------|---------------------|--------------------|------|----------------------|
| <i>PostgreSQL</i>       |                     |                     |                    |      |                      |
| pg < knex               | 20 [18–21]          | 27 [26–28]          | 1.40 [1.34–1.47]   | 1.00 | $5.0 \times 10^{-5}$ |
| knex < drizzle          | 27 [26–28]          | 35 [32–35]          | 1.26 [1.21–1.31]   | 0.96 | $5.0 \times 10^{-5}$ |
| drizzle < typeorm       | 35 [32–35]          | 50 [49–52]          | 1.49 [1.43–1.55]   | 1.00 | $5.0 \times 10^{-5}$ |
| typeorm < prisma        | 50 [49–52]          | 51 [51–53]          | 1.03 [1.00–1.05]   | 0.62 | 0.0538               |
| prisma < objection      | 51 [51–53]          | 57 [56–58]          | 1.10 [1.07–1.13]   | 0.92 | $5.0 \times 10^{-5}$ |
| objection < sequelize   | 57 [56–58]          | 60 [59–61]          | 1.05 [1.03–1.08]   | 0.86 | 0.0001               |
| sequelize < mikroorm    | 60 [59–61]          | 116 [113–119]       | 1.92 [1.87–1.98]   | 1.00 | $5.0 \times 10^{-5}$ |
| <i>MySQL</i>            |                     |                     |                    |      |                      |
| mysql2 < knex           | 28 [28–29]          | 30 [29–31]          | 1.06 [1.03–1.08]   | 0.80 | 0.0003               |
| knex < drizzle          | 30 [29–31]          | 47 [46–48]          | 1.57 [1.53–1.62]   | 1.00 | $5.0 \times 10^{-5}$ |
| drizzle < typeorm       | 47 [46–48]          | 57 [56–59]          | 1.22 [1.20–1.25]   | 1.00 | $5.0 \times 10^{-5}$ |
| typeorm < sequelize     | 57 [56–59]          | 67 [66–68]          | 1.17 [1.14–1.19]   | 0.96 | $5.0 \times 10^{-5}$ |
| sequelize < objection   | 67 [66–68]          | 69 [67–71]          | 1.03 [1.00–1.05]   | 0.66 | 0.0799               |
| objection < prisma      | 69 [67–71]          | 93 [89–95]          | 1.34 [1.30–1.39]   | 1.00 | $5.0 \times 10^{-5}$ |
| prisma < mikroorm       | 93 [89–95]          | 120 [118–128]       | 1.34 [1.29–1.38]   | 1.00 | $5.0 \times 10^{-5}$ |



bug invisible unless every layer’s output is checked against a reference, not a performance one.

- **OFFSET vs. keyset pagination.** An early, OFFSET-based design for the range-scan endpoint collapsed on MySQL at realistic offsets, which would have been misread as an access-layer weakness rather than what it is: a pagination-strategy cost that InnoDB pays more dearly for than PostgreSQL. Rewriting every adapter around keyset pagination (`WHERE id < cursor ORDER BY id DESC LIMIT n`) removed the confound.
- **Write-workload contamination.** The insert endpoint grows `posts` on every call; because engines and layers commit at different rates, later cells in a run would scan a differently sized table depending on run order rather than on layer identity, unless benchmark-inserted rows are deleted before every cell.
- **Query-formulation confounds.** A first attempt at the aggregation endpoint pre-aggregated *all* of `comments` before filtering to one author, so every request re-scanned the whole table regardless of which author was requested — a query-plan artifact masquerading as an access-layer effect, fixed by rewriting the query as a correlated subquery touching only the target author’s rows.

## 32 Statistical methods

This section details the tests summarized in the main text’s Analysis subsection. In every test the unit is the *run-level* statistic (one throughput or p99 value per repeated run, 25 per cell), never the individual request; all confidence intervals are percentile bootstraps resampling replicate indices (preserving the pairing) from a fixed seed (`mulberry32(0x57a75)`), so every resample is reproducible.

To test whether adjacently ranked layers are distinguishable, we compare each adjacent pair on the deep/nested fetch on their per-replicate throughput ratios with a two-sided paired sign-flip permutation test (20,000 permutations, so the smallest attainable  $p$  is  $1/(20,001)$ ), cross-checked with the Wilcoxon signed-rank test, and report the geometric-mean paired ratio with a paired bootstrap 95% CI and the paired dominance (the fraction of replicates the faster layer wins). Because p99 is reported at millisecond resolution and contains ties, the permutation test keeps zero log-ratios and the Wilcoxon test drops them with the standard tie correction. Because the seven adjacent pairs follow the *observed* ranking rather than a preregistered one, their significances are a secondary, descriptive robustness check, and we apply a Bonferroni correction as a conservatism check rather than claiming a confirmatory family.

The layer×engine interaction is tested per pattern with a blocked permutation test: for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference  $D_{\ell,i}$  and, under the null of no interaction, permute the layer labels of  $\{D_{\ell,i}\}$  *within* each replicate  $i$  (20,000 permutations); the statistic is the between-layer  $F$  of a randomized-block ANOVA on  $D$  with replicate as the block. Rank agreement across engines (Spearman, Kendall) is reported descriptively over the seven evaluated layers — systems under study, not a sample from a population — so we attach no population confidence intervals to those correlations.

For the pg-Prisma pair we additionally report a paired two-one-sided-tests (TOST) equivalence result against a  $\pm 5\%$  margin on the log-ratio. That margin is an author-selected smallest effect of practical interest for *this* study, not a general engineering threshold: at modest deployment scale a sub-5% throughput difference would not change a capacity-planning decision (for example, how many application instances to provision for a target load). We set it from that engineering consequence rather than from measurement noise; it happens also to sit within the run-to-run and day-to-day variation typical of a virtualized host (this study’s own run-to-run CV reaches 15.9% on the noisiest MySQL-insert cell, within 4.2% on the reads), but that coincidence is not the justification. At very large scale even 5% can correspond to many machines, so the margin should be reset per deployment. We chose it before the analysis but did not preregister it, and the equivalence conclusion is not sensitive to the exact value within a few percent.

### 33 Measurement details

This section records the measurement specifics summarized in the main text’s Measurement procedure.

**Warm-up length** The fifteen-second warm-up follows a separate cold-boot measurement for a fast, a middling, and the slowest PostgreSQL layer: every layer reached and sustained at least 95% of steady-state throughput within 12 s (Prisma by 5 s, native by 9 s, MikroORM by 12 s), absorbing JIT compilation, pool fill, and plan caching.

**Order invariance and completeness** Cell order was reshuffled independently within each replicate; a forward-versus-reversed order check on the write endpoint (5 replicates per direction) found no detectable history effect on the read cells (reversed-to-forward ratio 0.977–1.280, median 1.012; the wide tail is confined to the noisy MySQL inserts, where five replicates cannot separate order from write noise). Of the 450 dedicated write-endpoint boots, 449 completed normally; the exception was one Drizzle/MySQL write replicate whose server boot did not clear the post-reset health check, leaving

that cell with 24 of 25 runs (the runner records and skips such failures rather than retrying silently).

**Tail estimation** `autocannon` records a per-run latency histogram at millisecond resolution; even the slowest deep-fetch cells ( $\approx 480\text{--}516$  req/s over the 12 s window) complete  $\approx 5,800\text{--}6,200$  requests per run, so each run-level p99 is estimated from roughly 60 requests in its upper 1% tail, and the reported p99 is the median of the 25 run-level values.

**Resource and framework sampling** Server CPU and peak resident memory were sampled from `/proc` over the full process tree (parent plus descendants, so an in-process engine or spawned helper cannot escape measurement); a `perf_hooks` GC observer (event count, total pause time) and the 200 ms connection-pool sampler ran inside the server, polled through a `/stats` endpoint after each run. A fixed-payload `/baseline` endpoint returning a seed-realistic thread-shaped payload measured the shared Express-plus-serialization floor: 10,909 (PostgreSQL server) and 10,921 (MySQL server) req/s, roughly  $3\times$  the fastest deep-fetch cell, so the framework floor leaves between-layer spreads visible rather than compressed.

**Insert configuration** Each insert is an autocommit single-statement `INSERT` under each engine's default isolation level (PostgreSQL Read Committed, MySQL Repeatable Read); the transactional variant wraps a post and its five comments in one transaction. The default durability settings are PostgreSQL `fsync=on`, `synchronous_commit=on`, `full_page_writes=on` and MySQL `innodb_flush_log_at_trx_sync_binlog=1`, `log_bin=ON`.

## 34 Artifact reproducibility summary

Table [S31](#) summarizes what is needed to reproduce the study and which results are regenerated automatically; `REPRODUCE.md` in the replication package is the single runnable entry point.

Table S31: Artifact reproducibility summary: version and identifier, requirements, how to run, time and resources, and which results are reproduced automatically.

| Item                              | Detail                                                                                                                                                                                                                                                   |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Artifact & version                | <b>express-db-access-performance</b> , release v1.6.4 (git tag v1.6.4); the archive is a <b>git archive</b> of the tagged commit and contains every tracked file, including the raw data and the table generators.                                       |
| Persistent identifier             | Zenodo 10.5281/zenodo.21444624 (concept 10.5281/zenodo.21313858).                                                                                                                                                                                        |
| Software                          | Node.js 24.18.0, PostgreSQL 18.4, MySQL 9.7.1, <b>npm</b> . Reference path: conda user-space ( <b>conda-forge</b> ); the Docker path is a convenience and pins older engines (PostgreSQL 16 / MySQL 8.4), so it only approximates the published numbers. |
| Hardware & disk                   | Single Linux host (full host and version capture in Table S11); $\approx 1$ GB for the seeded databases.                                                                                                                                                 |
| Deterministic seed                | 2,000 authors / 100,000 posts / 1,000,000 comments from a fixed PRNG.                                                                                                                                                                                    |
| Smoke test ( $\approx 5$ –10 min) | <b>npm ci; npm run migrate &amp;&amp; npm run seed; npm run bench:quick; node bench/verify.mjs</b> — the byte-equivalence cross-check must print ALL MATCH.                                                                                              |
| Full campaign                     | Primary matrix ( $\approx$ hours): <b>INDEP=1 REPEATS=25 DURATION=12 WARMUP=15 node bench/runner.mjs <math>\rightarrow</math> results/raw.json</b> ; all secondary experiments add up to $\approx 25$ h wall-clock.                                      |
| Regenerate from raw               | $\approx$ minutes, <i>no database</i> : the committed generators rebuild every table and figure from <b>results/*.json</b> , then <b>npm run sync:tables</b> and <b>make</b> .                                                                           |
| Automatically reproduced          | <i>Every</i> table and figure, from the archived raw data; <b>results/checksums.sha256</b> verifies the 33 raw-data files. Reproducing the raw data itself requires the full campaign on the reference engines.                                          |
| Entry point                       | <b>REPRODUCE.md</b> (smoke test, full matrix, clean-room reproduction from the tarball); the per-table data-and-generator map is in <b>MANIFEST.md</b> .                                                                                                 |

Table S32: Scope of every experiment: the access pattern(s), engine(s), number of access-layer implementations, and repeated runs each covers. The *primary comparison* (throughput and closed-loop p99) is the full five-pattern, two-engine matrix; every other experiment is a limited condition, most restricted to the deep/nested fetch, and each conclusion is scoped accordingly. “Layers” counts implementations (the nine access layers plus, where present, the two tuned native baselines); a representative subset is used where a caption says so. Engine entries name where the experiment was run (pg is PostgreSQL-only, `mysql2` MySQL-only).

| Experiment                                                | Pattern(s)         | Engines   | Layers | Runs                 |
|-----------------------------------------------------------|--------------------|-----------|--------|----------------------|
| Primary matrix: throughput & closed-loop p99 (main paper) | all five           | PG, MySQL | 11     | 25 independent       |
| Same-SQL raw-path control (Table S14)                     | deep fetch         | PG, MySQL | 9      | median of 10         |
| Open-loop CO-corrected tail (Tables S6, S17)              | deep fetch         | PG, MySQL | 5      | rate sweep           |
| Utilization-controlled tail (Tables S21, S22)             | deep fetch         | PG, MySQL | 8      | median of 5          |
| Equal-CPU, 1/2/4 cores (Table S4), exploratory            | deep fetch         | PG        | 3      | median of 3          |
| Node-cluster multi-worker (Table S24)                     | deep fetch         | PG, MySQL | 3      | median of 3          |
| Pool-size sweep, 1–50 (Tables S7, S25)                    | deep fetch         | PG, MySQL | 4      | median of 3          |
| Mixed read/write, 90:10 & 70:30 (Table S26)               | deep fetch, insert | PG, MySQL | 5      | median of 5          |
| Relaxed vs default durability (Table S3)                  | insert             | PG, MySQL | 11     | 25 def. + 10 relaxed |
| Wait-event flush decomposition (Table S19)                | insert             | MySQL     | 3      | 4000 inserts/layer   |
| Transactional multi-statement write (Table S8)            | POST /threads      | PG        | 5      | median of 5          |
| Longer-window (60 s) tail (Table S23)                     | deep fetch         | PG, MySQL | 11     | 10 runs              |
| Post-restart cold cache (Table S20)                       | deep fetch         | PG, MySQL | 3      | vs primary median    |
| Alternative eager-loading (Table S18)                     | deep fetch         | PG, MySQL | 3      | median of 5          |